

Are coins product-aware?

Honest answer: **no, not properly**. Here's the gap and the fix.

What we built

- `Deal.coinsReward` — admin sets coins per purchase of a specific deal
- Fallback: $\text{commission} \times 10$ coins if no deal match

Why this is wrong for real affiliate

Real affiliate doesn't work "per product we promoted" — it works **per product the user actually bought**. Three scenarios that break the current model:

Scenario 1 — User clicks "Wireless Headphones" deal but buys a Bluetooth speaker on Amazon

We get commission for the Bluetooth speaker (different category, different rate). Current code awards the Headphones deal's `coinsReward` — wrong product. Merchant report row says `productId: BLUETOOTH_SPK, commission: ₹120` — we should use that.

Scenario 2 — User clicks one deal, buys 5 unrelated things

Merchant report has 5 rows, 5 different commissions. Current code can't handle multi-product orders from one click.

Scenario 3 — Commission rates vary wildly by category

Amazon: electronics 1%, fashion 8%, beauty 10%. A ₹1000 fashion order = ₹80 commission; a ₹1000 electronics order = ₹10. Same deal click, very different commission. Current model gives the same coins.

The real model

The merchant's commission column IS the source of truth. They calculated it based on the actual product, the actual category, the actual rate. We should derive coins from that, not from a pre-set deal value.

Cleaner formula:

$$\text{coins_for_order} = \text{commission} \times \text{PASS_THROUGH_RATE} \times \text{COINS_PER_RUPEE}$$

Where:

- `commission` — what the merchant says we earned for that specific order (row in report)
- `PASS_THROUGH_RATE` — what % of commission we give back to user as coins (e.g., 0.25 = 25%)
- `COINS_PER_RUPEE` — conversion rate (e.g., 10 coins per ₹1)

Worked example:

- Merchant report: order `MYN-X1`, commission `₹50`
- Pass-through 25%, rate 10 coins/₹ → user gets `50 × 0.25 × 10 = 125 coins`
- User redeems 125 coins = ₹12.50 to UPI

This is **product-aware** because the merchant did the per-product math for us. We just split the commission with the user.

What `Deal.coinsReward` should become

Not the primary mechanism. Two reasonable uses:

1. **Promotional override** — “this deal pays a flat 200 coins regardless of what they actually buy from this merchant”
2. **Drop it entirely** — the formula handles everything; per-deal config adds complexity for no real benefit

What needs to change

Change	Why
Admin settings: <code>passThroughPercent</code> , <code>coinsPerRupee</code>	One place to tune the whole economy
Importer formula → <code>commission × passThroughPercent × coinsPerRupee</code>	Product-aware via the report
<code>Deal.coinsReward</code> → optional override (used only with click → deal match)	Backward compat + promo support
Reports Inbox preview: “We keep ₹X, user gets Y coins ($\approx$ ₹Z)” per row	Transparency at import time
Wallet Ops Settings page — adjust pass-through % + conversion rate, live preview	Admin can balance giving vs keeping

Concrete numbers

At 25% pass-through + 10 coins per ₹:

Product bought	Commission to us	Coins to user	User's ₹ equiv	We keep
₹2000 fashion (Myntra, 8%)	₹160	400	₹40	₹120 (75%)
₹500 electronics (Amazon, 1%)	₹5	12	₹1.20	₹3.80 (76%)
₹1500 beauty (Nykaa, 10%)	₹150	375	₹37.50	₹112.50 (75%)

User always gets ~25% of what we earn on their purchase. Their reward scales with what they actually bought, not what we put in front of them.

Scope to fix

Clean ~1 day change:

1. Settings model + admin settings page (pass-through %, conversion rate) — 3h
2. Importer formula swap — 1h

3. Reports Inbox preview shows the per-row breakdown — 1h
4. Deal.coinsReward demoted to optional override + clarified in admin form — 1h
5. Update the client PDF to reflect the corrected model — 30m
6. End-to-end test — 1h

This is the right time to fix — before any real merchant report goes through.